

Выселили соседей. Оставили вам сервер.

Bare Metal от RUVDS — физическая машина без гипервизора, все ресурсы в вашем распоряжении.

Не готовы брать на месяц? Арендуйте сервер на 1 день — **от 661,5 Р.**



Реклама ООО «ИТ-ФИНАНС» ERID: LUNKAR35



512K+

Охват за 30 дней

RUVDS.com

4 121,11 Рейтинг

159 361 Подписчики

Подписаться



ru_vds

28 апр 2017 в 16:55

Bash-скрипты, часть 7: sed и обработка текстов



9 мин



691K

Блог компании RUVDS.com, Настройка Linux*, Серверное администрирование*

Перевод

Автор оригинала: Администратор likegeeks.com

Bash-скрипты: начало

Bash-скрипты, часть 2: циклы

Bash-скрипты, часть 3: параметры и ключи командной строки

Bash-скрипты, часть 4: ввод и вывод

Bash-скрипты, часть 5: сигналы, фоновые задачи, управление сценариями

Bash-скрипты, часть 6: функции и разработка библиотек

Bash-скрипты, часть 7: sed и обработка текстов

Bash-скрипты, часть 8: язык обработки данных awk

Bash-скрипты, часть 9: регулярные выражения

Bash-скрипты, часть 10: практические примеры

Bash-скрипты, часть 11: expect и автоматизация интерактивных утилит

В прошлый раз мы говорили о функциях в bash-скриптах, в частности, о том, как вызывать их из командной строки. Наша сегодняшняя тема — весьма полезный инструмент для обработки строковых данных — утилита Linux, которая называется sed. Её часто используют для работы с текстами, имеющими вид лог-файлов, конфигурационных и других файлов.



Если вы, в bash-скриптах, каким-то образом обрабатываете данные, вам не мешает знакомство с инструментами sed и gawk. Тут мы сосредоточимся на sed и на работе с текстами, так как это – очень важный шаг в нашем путешествии по бескрайним просторам разработки bash-скриптов.

Habrahabr10

Промо-код для скидки в 10% на наши виртуальные сервера

Сейчас мы разберём основы работы с sed, а так же рассмотрим более трёх десятков примеров использования этого инструмента.

Основы работы с sed

Утилиту sed называют потоковым текстовым редактором. В интерактивных текстовых редакторах, наподобие nano, с текстами работают, используя клавиатуру, редактируя файлы, добавляя, удаляя или изменяя тексты. Sed позволяет редактировать потоки данных, основываясь на заданных разработчиком наборах правил. Вот как выглядит схема вызова этой команды:

```
$ sed options file
```

Объяснить с 📖

По умолчанию sed применяет указанные при вызове правила, выраженные в виде набора команд, к STDIN . Это позволяет передавать данные непосредственно sed.

Например, так:

```
$ echo "This is a test" | sed 's/test/another test/'
```

Объяснить с 📖

Вот что получится при выполнении этой команды.

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ echo "This is a test" | sed 's/test/another test/'
This is a another test
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Простой пример вызова sed

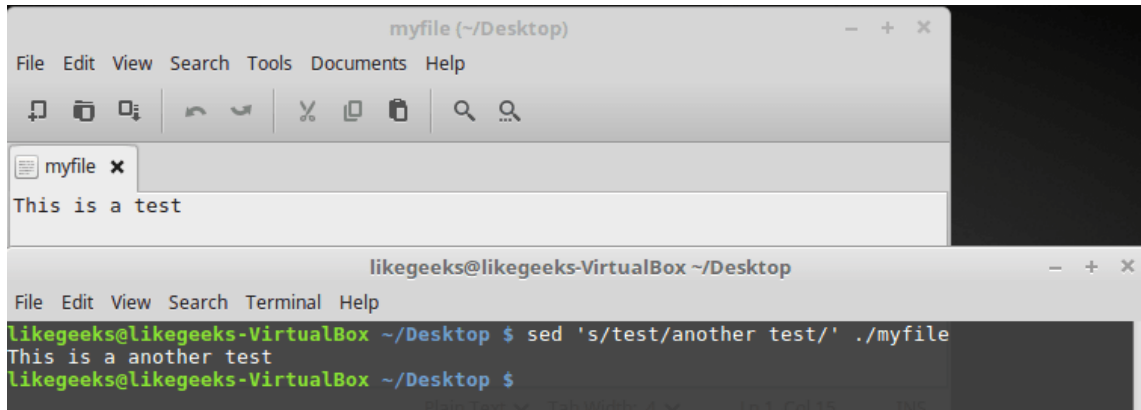
В данном случае sed заменяет слово «test» в строке, переданной для обработки, словами «another test». Для оформления правила обработки текста, заключённого в кавычки, используются прямые слэши. В нашем случае применена команда вида `s/pattern1/pattern2/` . Буква «s» – это сокращение слова «substitute», то есть – перед нами команда замены. Sed, выполняя эту

команду, просмотрит переданный текст и заменит найденные в нём фрагменты (о том – какие именно, поговорим ниже), соответствующие `pattern1`, на `pattern2`.

Выше приведён примитивный пример использования `sed`, нужный для того, чтобы ввести вас в курс дела. На самом деле, `sed` можно применять в гораздо более сложных сценариях обработки текстов, например – для работы с файлами.

Ниже показан файл, в котором содержится фрагмент текста, и результаты его обработки такой командой:

```
$ sed 's/test/another test' ./myfile
```

[Объяснить с](#) 

Текстовый файл и результаты его обработки

Здесь применён тот же подход, который мы использовали выше, но теперь `sed` обрабатывает текст, хранящийся в файле. При этом, если файл достаточно велик, можно заметить, что `sed` обрабатывает данные порциями и выводит то, что обработано, на экран, не дожидаясь обработки всего файла.

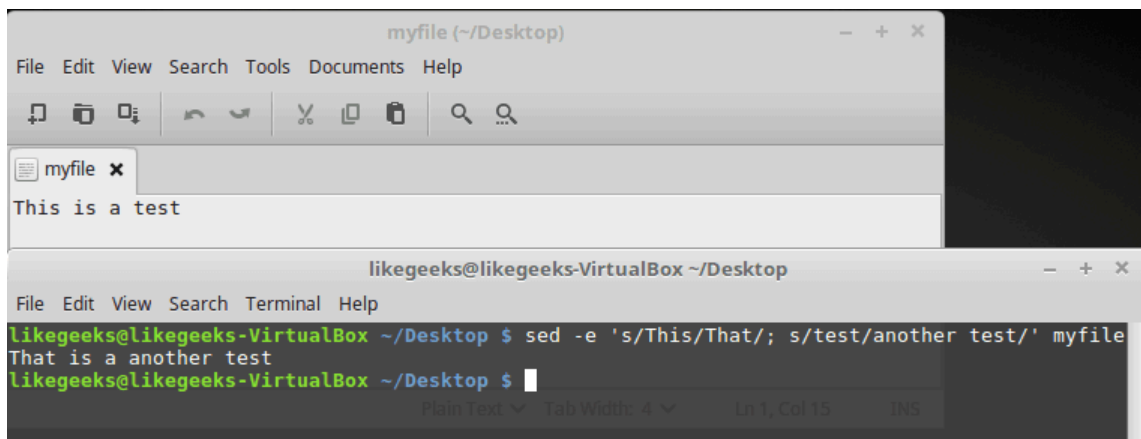
`Sed` не меняет данные в обрабатываемом файле. Редактор читает файл, обрабатывает прочитанное, и отправляет то, что получилось, в `STDOUT`. Для того, чтобы убедиться в том, что исходный файл не изменился, достаточно, после того, как он был передан `sed`, открыть его. При необходимости вывод `sed` можно перенаправить в файл, возможно – перезаписать старый файл. Если вы знакомы с одним из предыдущих материалов этой серии, где речь идёт о перенаправлении потоков ввода и вывода, вы вполне сможете это сделать.

Выполнение наборов команд при вызове `sed`

Для выполнения нескольких действий с данными, используйте ключ `-e` при вызове `sed`. Например, вот как организовать замену двух фрагментов текста:

```
$ sed -e 's/This/That/; s/test/another test/' ./myfile
```

[Объяснить с](#) 



Использование ключа `-e` при вызове `sed`

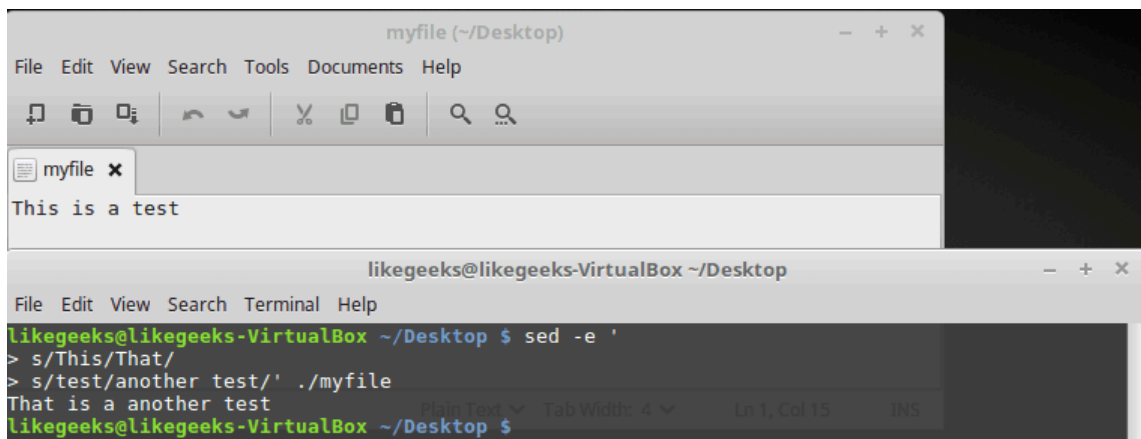
К каждой строке текста из файла применяются обе команды. Их нужно разделить точкой с запятой, при этом между окончанием команды и точкой с запятой не должно быть пробела.

Для ввода нескольких шаблонов обработки текста при вызове `sed`, можно, после ввода первой одиночной кавычки, нажать Enter, после чего вводить каждое правило с новой строки, не забыв о закрывающей кавычке:

```
$ sed -e '  
> s/This/That/  
> s/test/another test/' ./myfile
```

Объяснить с

Вот что получится после того, как команда, представленная в таком виде, будет выполнена.



Другой способ работы с `sed`

Чтение команд из файла

Если имеется множество команд `sed`, с помощью которых надо обработать текст, обычно удобнее всего предварительно записать их в файл. Для того, чтобы указать `sed` файл, содержащий команды, используют ключ `-f`:

Вот содержимое файла `mycommands`:

```
s/This/That/  
s/test/another test/
```

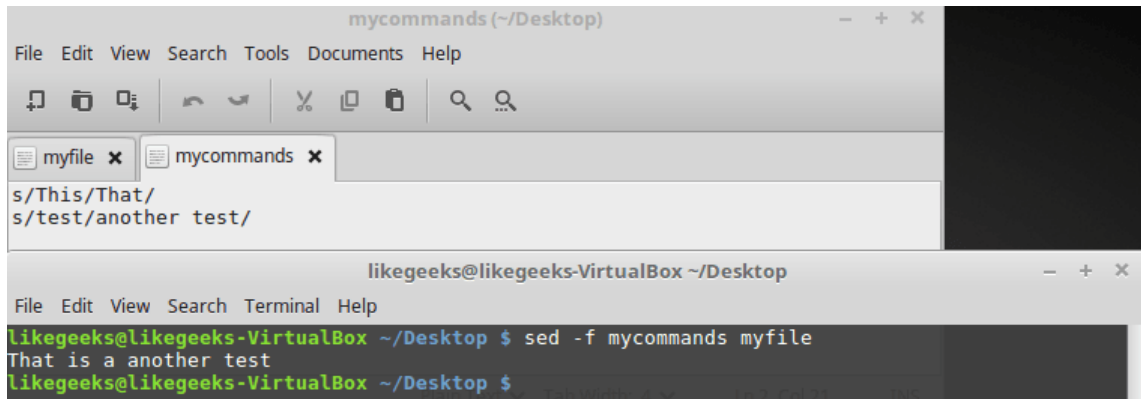
Объяснить с

Вызовем sed, передав редактору файл с командами и файл для обработки:

```
$ sed -f mycommands myfile
```

[Объяснить с](#)

Результат при вызове такой команды аналогичен тому, который получался в предыдущих примерах.



Использование файла с командами при вызове sed

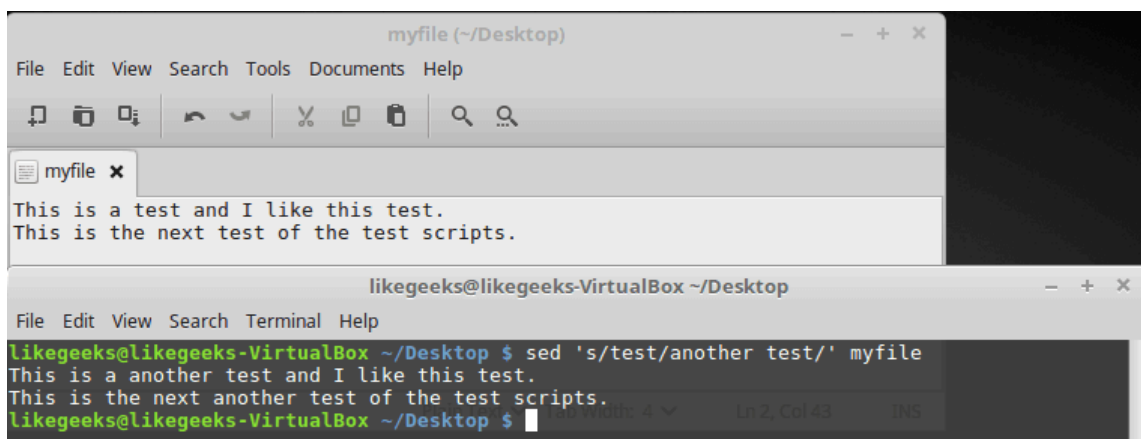
Флаги команды замены

Внимательно посмотрите на следующий пример.

```
$ sed 's/test/another test/' myfile
```

[Объяснить с](#)

Вот что содержится в файле, и что будет получено после его обработки sed.



Исходный файл и результаты его обработки

Команда замены нормально обрабатывает файл, состоящий из нескольких строк, но заменяются только первые вхождения искомого фрагмента текста в каждой строке. Для того, чтобы заменить все вхождения шаблона, нужно использовать соответствующий флаг.

Схема записи команды замены при использовании флагов выглядит так:

```
s/pattern/replacement/flags
```

[Объяснить с](#)

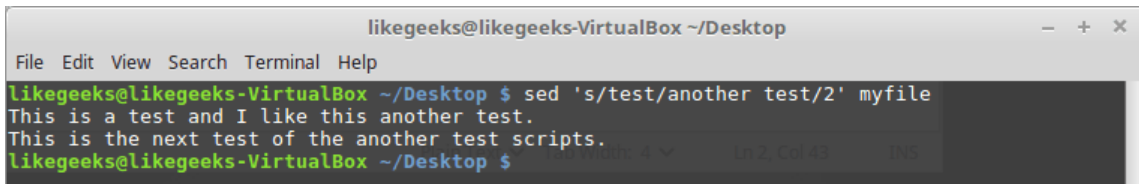
Выполнение этой команды можно модифицировать несколькими способами.

- При передаче номера учитывается порядковый номер вхождения шаблона в строку, заменено будет именно это вхождение.
- Флаг `g` указывает на то, что нужно обработать все вхождения шаблона, имеющиеся в строке.
- Флаг `r` указывает на то, что нужно вывести содержимое исходной строки.
- Флаг вида `w file` указывает команде на то, что нужно записать результаты обработки текста в файл.

Рассмотрим использование первого варианта команды замены, с указанием позиции заменяемого вхождения искомого фрагмента:

```
$ sed 's/test/another test/2' myfile
```

Объяснить с 



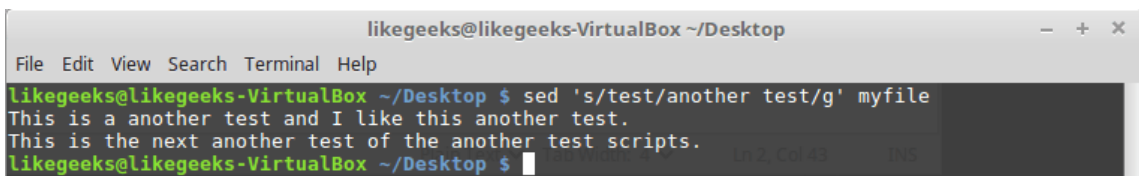
```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed 's/test/another test/2' myfile
This is a test and I like this another test.
This is the next test of the another test scripts.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Вызов команды замены с указанием позиции заменяемого фрагмента

Тут мы указали, в качестве флага замены, число 2. Это привело к тому, что было заменено лишь второе вхождение искомого шаблона в каждой строке. Теперь опробуем флаг глобальной замены – `g` :

```
$ sed 's/test/another test/g' myfile
```

Объяснить с 



```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed 's/test/another test/g' myfile
This is a another test and I like this another test.
This is the next another test of the another test scripts.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

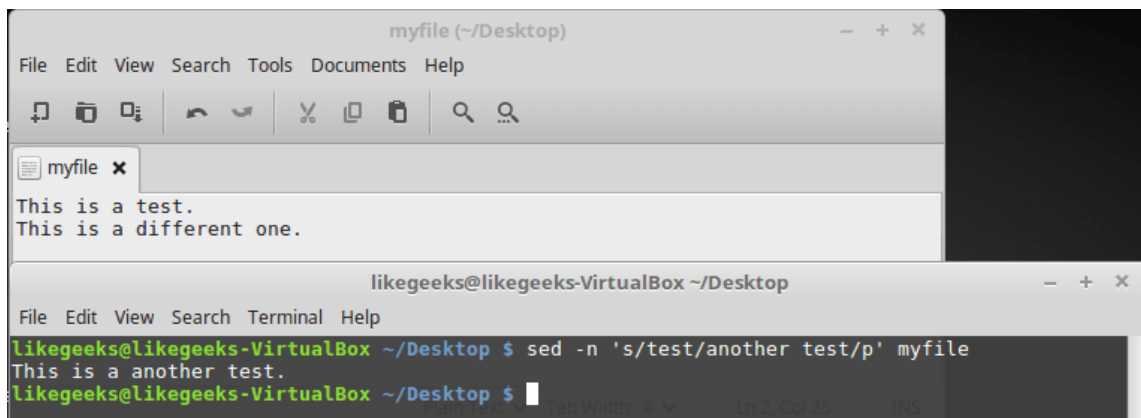
Глобальная замена

Флаг команды замены `p` позволяет выводить строки, в которых найдены совпадения, при этом ключ `-n`, указанный при вызове `sed`, подавляет обычный вывод:

```
$ sed -n 's/test/another test/p' myfile
```

Объяснить с 

Как результат, при запуске `sed` в такой конфигурации на экран выводятся лишь строки (в нашем случае – одна строка), в которых найден заданный фрагмент текста.

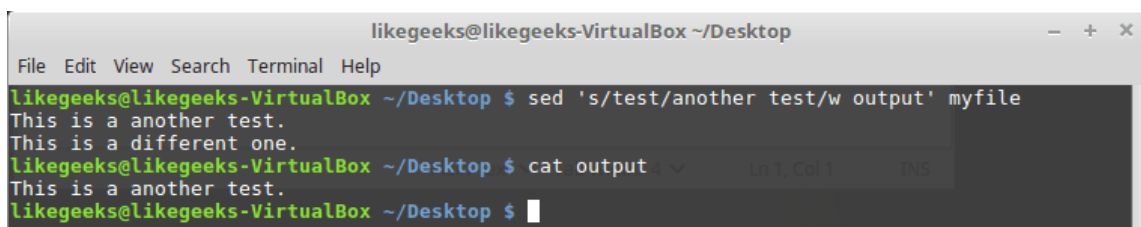


Использование флага команды замены *p*

Вспользуемся флагом *w* , который позволяет сохранить результаты обработки текста в файл:

```
$ sed 's/test/another test/w output' myfile
```

Объяснить с



Сохранение результатов обработки текста в файл

Хорошо видно, что в ходе работы команды данные выводятся в *STDOUT*, при этом обработанные строки записываются в файл, имя которого указано после *w* .

Символы-разделители

Представьте, что нужно заменить */bin/bash* на */bin/csh* в файле */etc/passwd* . Задача не такая уж и сложная:

```
$ sed 's/\bin/bash/\bin/csh/' /etc/passwd
```

Объяснить с

Однако, выглядит всё это не очень-то хорошо. Всё дело в том, что так как прямые слэши используются в роли символов-разделителей, такие же символы в передаваемых *sed* строках приходится экранировать. В результате страдает читаемость команды.

К счастью, *sed* позволяет нам самостоятельно задавать символы-разделители для использования их в команде замены. Разделителем считается первый символ, который будет встречен после *s* :

```
$ sed 's!/bin/bash!/bin/csh!' /etc/passwd
```

Объяснить с

В данном случае в качестве разделителя использован восклицательный знак, в результате код легче читать и он выглядит куда опрятнее, чем прежде.

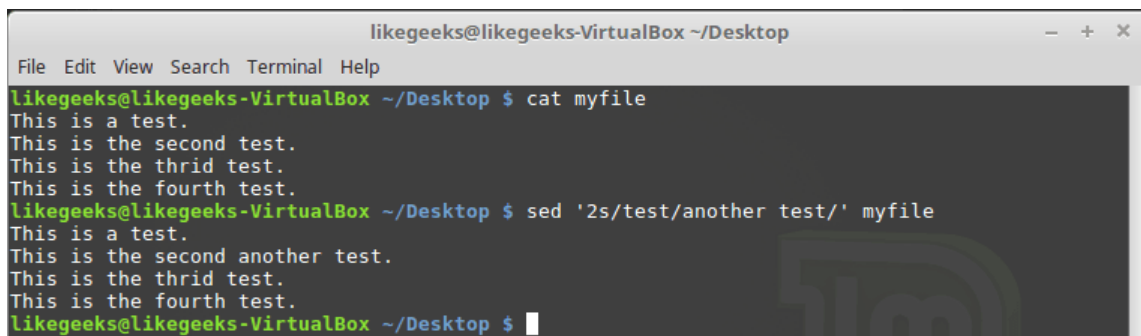
Выбор фрагментов текста для обработки

До сих пор мы вызывали `sed` для обработки всего переданного редактору потока данных. В некоторых случаях с помощью `sed` надо обработать лишь какую-то часть текста – некую конкретную строку или группу строк. Для достижения такой цели можно воспользоваться двумя подходами:

- Задать ограничение на номера обрабатываемых строк.
- Указать фильтр, соответствующие которому строки нужно обработать.

Рассмотрим первый подход. Тут допустимо два варианта. Первый, рассмотренный ниже, предусматривает указание номера одной строки, которую нужно обработать:

```
$ sed '2s/test/another test/' myfile
```

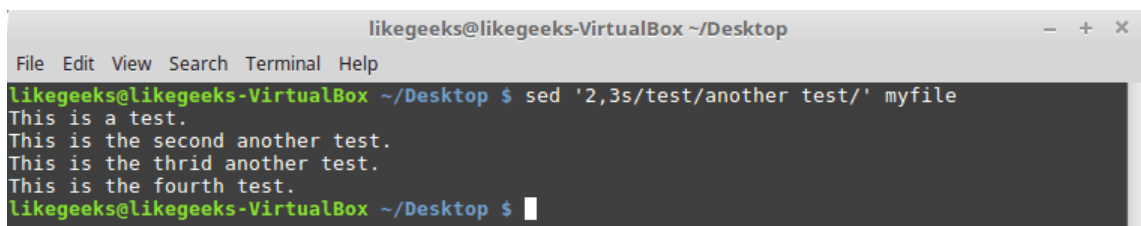
[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '2s/test/another test/' myfile
This is a test.
This is the second another test.
This is the thrid test.
This is the fourth test.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Обработка только одной строки, номер который задан при вызове `sed`

Второй вариант – диапазон строк:

```
$ sed '2,3s/test/another test/' myfile
```

[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '2,3s/test/another test/' myfile
This is a test.
This is the second another test.
This is the thrid another test.
This is the fourth test.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Обработка диапазона строк

Кроме того, можно вызвать команду замены так, чтобы файл был обработан начиная с некоей строки и до конца:

```
$ sed '2,$s/test/another test/' myfile
```

[Объяснить с](#)


```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '2,$s/test/another test/' myfile
This is a test.
This is the second another test.
This is the thrid another test.
This is the fourth another test.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Обработка файла начиная со второй строки и до конца

Для того, чтобы обрабатывать с помощью команды замены только строки, соответствующие заданному фильтру, команду надо вызвать так:

```
$ sed '/likegeeks/s/bash/csh/' /etc/passwd
```

Объяснить с 

По аналогии с тем, что было рассмотрено выше, шаблон передаётся перед именем команды `s`.

```
rtkit:x:119:127:RealtimeKit,,,:/proc:/bin/false
saned:x:120:128:./var/lib/saned:/bin/false
usbmux:x:121:46:usbmux daemon,,,:/var/lib/usbmux:/bin/false
likegeeks:x:1000:1000:likegeeks,,,:/home/likegeeks:/bin/csh
vboxadd:x:999:1:./var/run/vboxadd:/bin/false
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Обработка строк, соответствующих фильтру

Тут мы использовали очень простой фильтр. Для того, чтобы в полной мере раскрыть возможности данного подхода, можно воспользоваться регулярными выражениями. О них мы поговорим в одном из следующих материалов этой серии.

Удаление строк

Утилита `sed` годится не только для замены одних последовательностей символов в строках на другие. С её помощью, а именно, используя команду `d`, можно удалять строки из текстового потока.

Вызов команды выглядит так:

```
$ sed '3d' myfile
```

Объяснить с 

Мы хотим, чтобы из текста была удалена третья строка. Обратите внимание на то, что речь не идёт о файле. Файл останется неизменным, удаление отразится лишь на выводе, который сформирует `sed`.

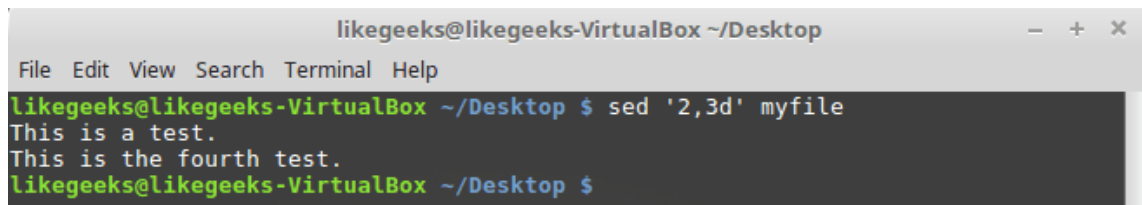
```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '3d' myfile
This is a test.
This is the second test.
This is the fourth test.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Удаление третьей строки

Если при вызове команды `d` не указать номер удаляемой строки, удалены будут все строки потока.

Вот как применить команду `d` к диапазону строк:

```
$ sed '2,3d' myfile
```

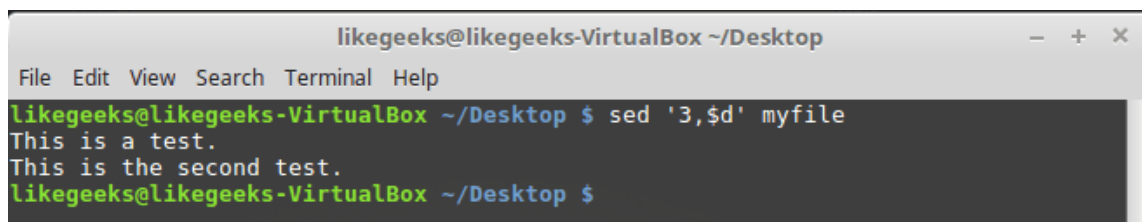
[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '2,3d' myfile
This is a test.
This is the fourth test.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Удаление диапазона строк

А вот как удалить строки, начиная с заданной – и до конца файла:

```
$ sed '3,$d' myfile
```

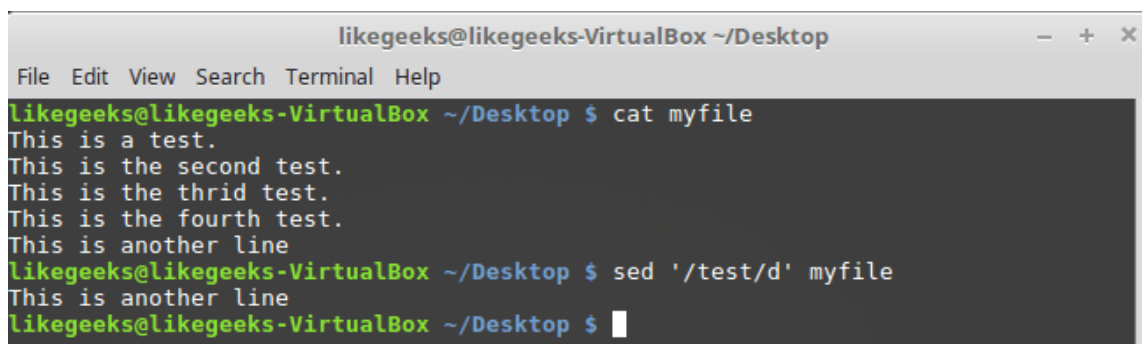
[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '3,$d' myfile
This is a test.
This is the second test.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Удаление строк до конца файла

Строки можно удалять и по шаблону:

```
$ sed '/test/d' myfile
```

[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '/test/d' myfile
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

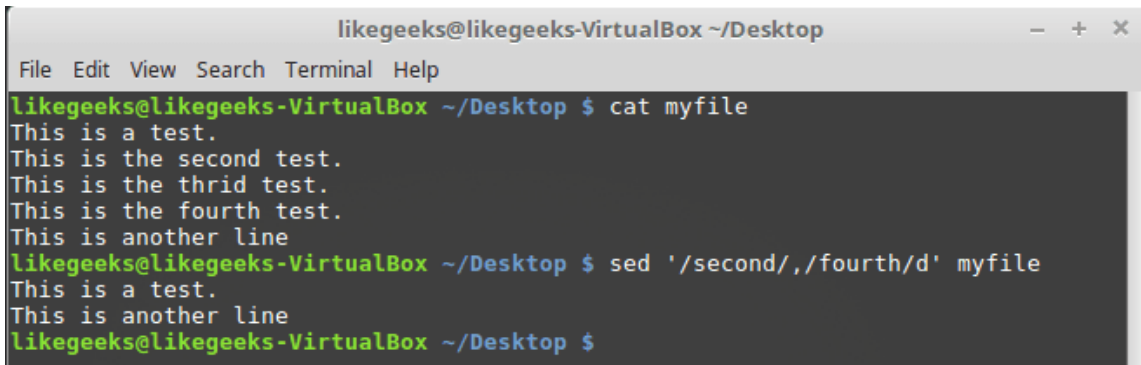
Удаление строк по шаблону

При вызове `d` можно указывать пару шаблонов – будут удалены строки, в которых встретится

шаблон, и те строки, которые находятся между ними:

```
$ sed '/second/,/fourth/d' myfile
```

Объяснить с 



```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '/second/,/fourth/d' myfile
This is a test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Удаление диапазона строк с использованием шаблонов

Вставка текста в поток

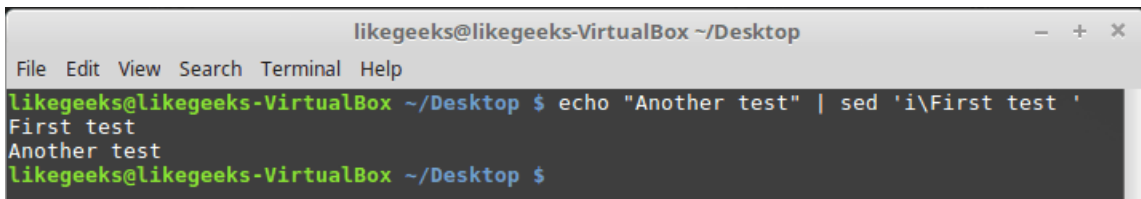
С помощью sed можно вставлять данные в текстовый поток, используя команды `i` и `a`:

- Команда `i` добавляет новую строку перед заданной.
- Команда `a` добавляет новую строку после заданной.

Рассмотрим пример использования команды `i`:

```
$ echo "Another test" | sed 'i\First test '
```

Объяснить с 



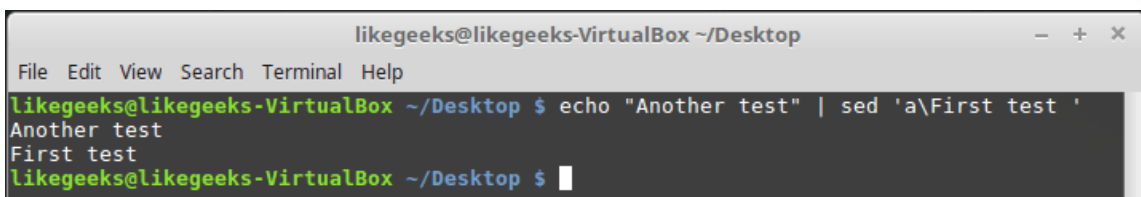
```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ echo "Another test" | sed 'i\First test '
First test
Another test
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Команда `i`

Теперь взглянем на команду `a`:

```
$ echo "Another test" | sed 'a\First test '
```

Объяснить с 



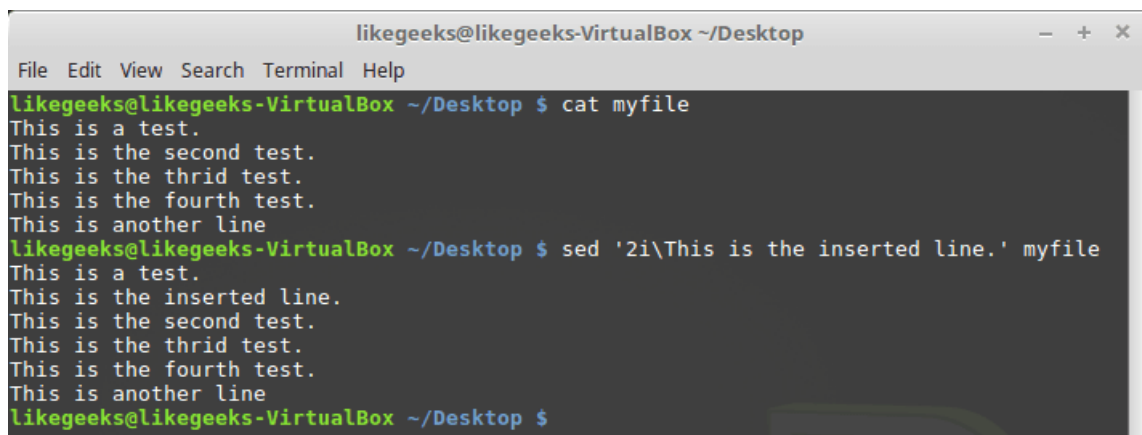
```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ echo "Another test" | sed 'a\First test '
Another test
First test
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Команда `a`

Как видно, эти команды добавляют текст до или после данных из потока. Что если надо добавить строку где-нибудь посередине?

Тут нам поможет указание номера опорной строки в потоке, или шаблона. Учтите, что адресация строк в виде диапазона тут не подойдёт. Вызовем команду `i`, указав номер строки, перед которой надо вставить новую строку:

```
$ sed '2i\This is the inserted line.' myfile
```

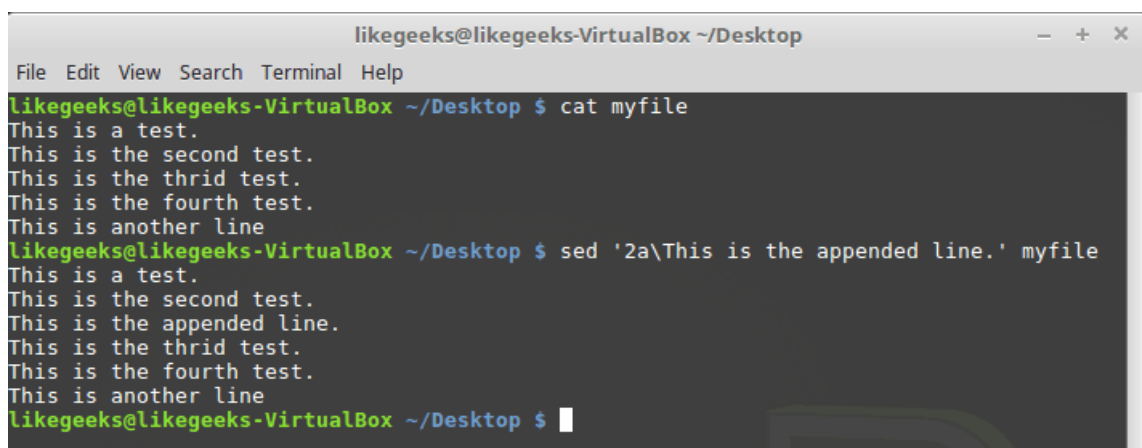
[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '2i\This is the inserted line.' myfile
This is a test.
This is the inserted line.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Команда `i` с указанием номера опорной строки

Прделаем то же самое с командой `a`:

```
$ sed '2a\This is the appended line.' myfile
```

[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '2a\This is the appended line.' myfile
This is a test.
This is the second test.
This is the appended line.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

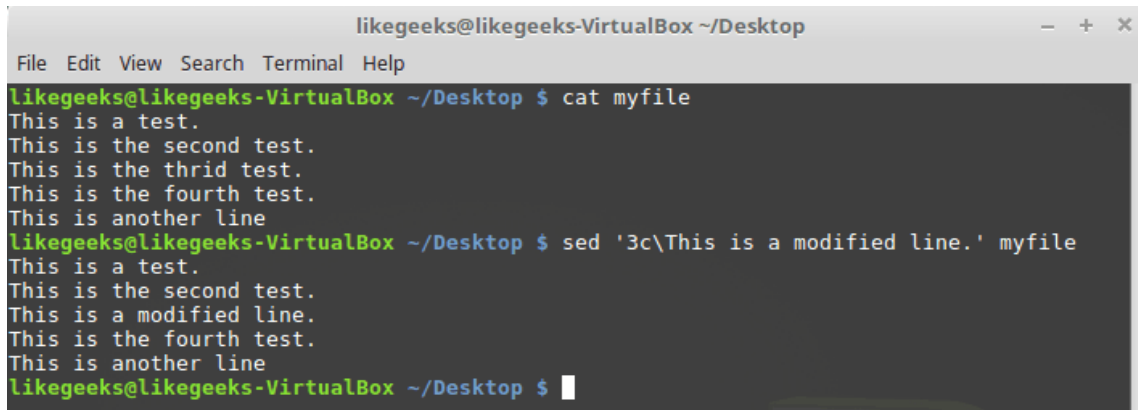
Команда `a` с указанием номера опорной строки

Обратите внимание на разницу в работе команд `i` и `a`. Первая вставляет новую строку до указанной, вторая – после.

Замена строк

Команда `s` позволяет изменить содержимое целой строки текста в потоке данных. При её вызове нужно указать номер строки, вместо которой в поток надо добавить новые данные:

```
$ sed '3c\This is a modified line.' myfile
```

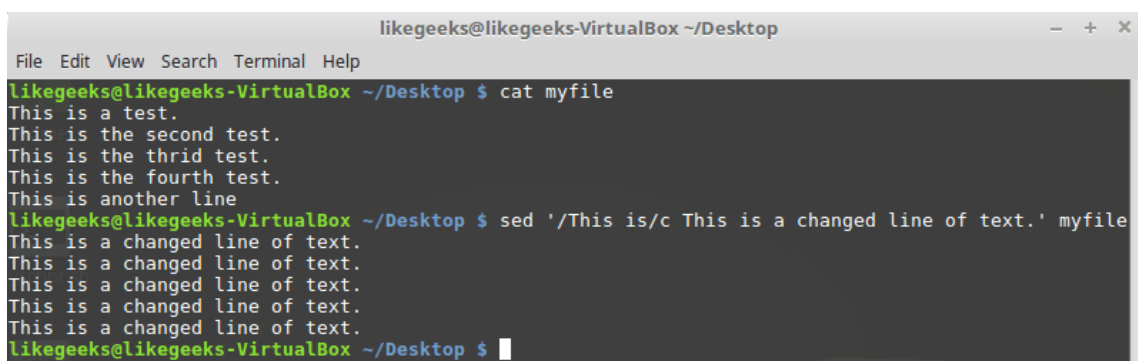
[Объяснить с](#) 

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '3c\This is a modified line.' myfile
This is a test.
This is the second test.
This is a modified line.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Замена строки целиком

Если воспользоваться при вызове команды шаблоном в виде обычного текста или регулярного выражения, заменены будут все соответствующие шаблону строки:

```
$ sed '/This is/c This is a changed line of text.' myfile
```

[Объяснить с](#) 

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '/This is/c This is a changed line of text.' myfile
This is a changed line of text.
This is a changed line of text.
This is a changed line of text.
This is a changed line of text.
This is a changed line of text.
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Замена строк по шаблону

Замена символов

Команда `y` работает с отдельными символами, заменяя их в соответствии с переданными ей при вызове данными:

```
$ sed 'y/123/567/' myfile
```

[Объяснить с](#) 

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.1
This is the second test.2
This is the thrid test.3
This is the fourth test.4
This is another line5
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed 'y/123/567/' myfile
This is a test.5
This is the second test.6
This is the thrid test.7
This is the fourth test.4
This is another line5
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Замена символов

Используя эту команду, нужно учесть, что она применяется ко всему текстовому потоку, ограничить её конкретными вхождениями символов нельзя.

Вывод номеров строк

Если вызвать sed, используя команду =, утилита выведет номера строк в потоке данных:

```
$ sed '=' myfile
```

[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '=' myfile
1
This is a test.
2
This is the second test.
3
This is the thrid test.
4
This is the fourth test.
5
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Вывод номеров строк

Потоковый редактор вывел номера строк перед их содержимым.

Если передать этой команде шаблон и воспользоваться ключом sed -n, выведены будут только номера строк, соответствующих шаблону:

```
$ sed -n '/test/=' myfile
```

[Объяснить с](#)

```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed -n '/test/=' myfile
1
2
3
4
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Вывод номеров строк, соответствующих шаблону

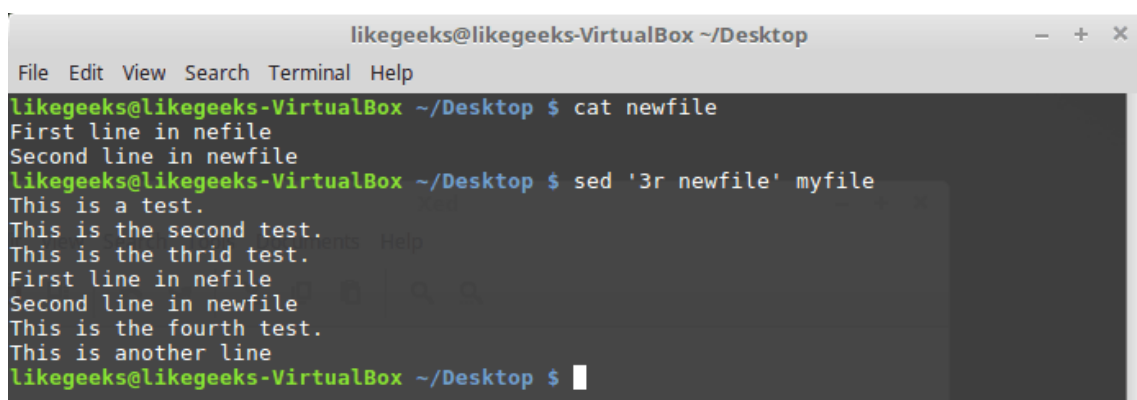
Чтение данных для вставки из файла

Выше мы рассматривали приёмы вставки данных в поток, указывая то, что надо вставить, прямо при вызове `sed`. В качестве источника данных можно воспользоваться и файлом. Для этого служит команда `r`, которая позволяет вставлять в поток данные из указанного файла. При её вызове можно указать номер строки, после которой надо вставить содержимое файла, или шаблон.

Рассмотрим пример:

```
$ sed '3r newfile' myfile
```

Объяснить с 



```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat newfile
First line in newfile
Second line in newfile
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '3r newfile' myfile
This is a test.
This is the second test.
This is the thrid test.
First line in newfile
Second line in newfile
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

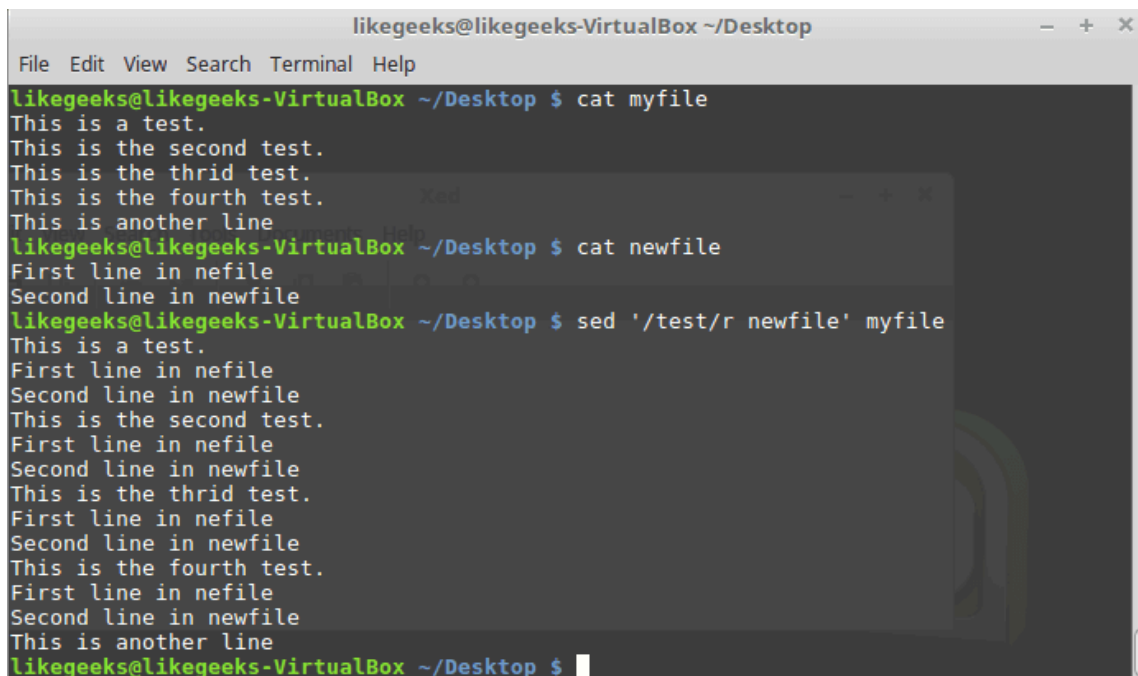
Вставка в поток содержимого файла

Тут содержимое файла `newfile` было вставлено после третьей строки файла `myfile`.

Вот что произойдёт, если применить при вызове команды `r` шаблон:

```
$ sed '/test/r newfile' myfile
```

Объяснить с 



```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat newfile
First line in nefile
Second line in newfile
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '/test/r newfile' myfile
This is a test.
First line in nefile
Second line in newfile
This is the second test.
First line in nefile
Second line in newfile
This is the thrid test.
First line in nefile
Second line in newfile
This is the fourth test.
First line in nefile
Second line in newfile
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Использование шаблона при вызове команды `g`

Содержимое файла будет вставлено после каждой строки, соответствующей шаблону.

Пример

Представим себе такую задачу. Есть файл, в котором имеется некая последовательность символов, сама по себе бессмысленная, которую надо заменить на данные, взятые из другого файла. А именно, пусть это будет файл `newfile`, в котором роль указателя места заполнения играет последовательность символов `DATA`. Данные, которые нужно подставить вместо `DATA`, хранятся в файле `data`.

Решить эту задачу можно, воспользовавшись командами `r` и `d` потокового редактора `sed`:

```
$ Sed '/DATA>/ {
r newfile
d}' myfile
```

Объяснить с 


```
likegeeks@likegeeks-VirtualBox ~/Desktop
File Edit View Search Terminal Help
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat newfile
First part of the data
Second part of the data
likegeeks@likegeeks-VirtualBox ~/Desktop $ cat myfile
This is a test.
This is the second test.
This is the thrid test.
DATA
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $ sed '/DATA/ {
r newfile
d}' myfile
This is a test.
This is the second test.
This is the thrid test.
First part of the data
Second part of the data
This is the fourth test.
This is another line
likegeeks@likegeeks-VirtualBox ~/Desktop $
```

Замена указателя места заполнения на реальные данные

Как видите, вместо заполнителя DATA sed добавил в выходной поток две строки из файла data .

Итоги

Сегодня мы рассмотрели основы работы с потоковым редактором sed. На самом деле, sed – это огромнейшая тема. Его изучение вполне можно сравнить с изучением нового языка программирования, однако, поняв основы, вы сможете освоить sed на любом необходимом вам уровне. В результате ваши возможности по обработке с его помощью текстов будет ограничивать лишь воображение.

На сегодня это всё. В следующий раз поговорим о языке обработки данных awk.

Habrahabr10

Промо-код для скидки в 10% на наши виртуальные сервера

Уважаемые читатели! А вы пользуетесь sed в повседневной работе? Если да – поделитесь пожалуйста опытом.

Теги: Linux, Bash, сценарий командной строки, Bash-скрипт, sed, обработка текстов

Хабы: Блог компании RUVDS.com, Настройка Linux, Серверное администрирование

+14

452



34



512K+

Охват за 30 дней

RUVDS.com

Подписаться

Telegram ВКонтакте X



128K+

Охват за 30 дней

↑ 617 ↓

Карма

@ru_vds

Пользователь

Подписаться



Комментарии 34



redfs

28 апр 2017 в 17:29

В следующий раз поговорим о языке обработки данных awk

А какое отношение sed и awk имеют к bash? То ли автор оригинального текста просто напихал все в одну кучу для создания объема, то ли назвал статью неправильно.

↑ +1 ↓

Ответить



НЛО прилетело и опубликовало эту надпись здесь



redfs

28 апр 2017 в 18:56

для баш-скриптов на любую тему

Так и при чем тут bash? Написал бы – для шелл-скриптов. awk и sed безусловно – супер инструменты. Но это **отдельные** темы для отдельных статей. Привязывать их теме bash – только путать новичков. Иногда правильнее вовремя остановиться. imho.

↑ 0 ↓

Ответить



madkite

3 мая 2017 в 23:49

.....



Найдите курс под свою цель

внешние программы.

↑ 0 ↓

Ответить



redfs

4 мая 2017 в 00:27

А вы замените слово bash в названии этой статьи на sh|ash|dash|zsh|ksh|csh|tcsh... на выбор. Ничего в самой статье не изменится, верно?

По поводу скудности – это зависит от подхода... К примеру. Автор этого цикла статей (как верно заметил @ ghostinushanka) ничего не написал про globbing вообще и *extended globbing* в bash в частности. Вот уж совсем не скудная и близкая к реальности тема.

↑ 0 ↓

Ответить



madkite

4 мая 2017 в 19:46

Так "bash scripting" – это название всего цикла. Это стандартная практика пихать в книги/циклы статей сопутствующий материал, только косвенно относящийся к тематике, просто указывая во введении что-то типа: "Если вы, в bash-скриптах, каким-то образом обрабатываете данные, вам не помешает знакомство с инструментами sed и gawk".

↑ 0 ↓

Ответить



ghostinushanka

28 апр 2017 в 19:03

Sed и Awk безусловно супер инструменты, но только к башу они отношения не имеют никакого. Оба являются язык программирования и посему не имеет значения из какого шелла они вызывается, ровно как и скрипты на python/perl/ruby/[insert your preferred language]. С таким же успехом эту статью можно было назвать «часть 7: perl и обработка текстов» и точно так же скармливать ему текст.

Когда мы говорим о bash программировании и обработке текстов, мы говорим в первую очередь о `parameter expansion`. И во многих случаях не надо ничего «извне».

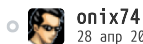
Посему согласен с @ redfs

Edit:

Чесслово не подглядывал когда писал фразу «*Sed и Awk безусловно супер инструменты*»



Ответить



onix74

28 апр 2017 в 19:17



Вообще, скорее, потоковые редакторы. sed, например, – Stream Editor. Но они на столько мощные, что уже можно и программировать. Вот, например, Тетрис. К bash, конечно, же они прямого отношения не имеют.



Ответить



indestructable

9 апр 2021 в 16:38



Спасибо за ссылку на прекрасную статью про `parameter expansion` и супер сайт



Ответить



grossws

28 апр 2017 в 19:58



По теме же поста – я бы еще флаг `-i` упомянул: открыть файл, провести редактирование, результат записать в тот же файл.

Как он с симлинками работает знаете? Я б не рекомендовал его использовать в общем случае без осознания связанных проблем.



Ответить



g4xd

4 мая 2017 в 20:10



```
sed -i.bak '/text/s/patern1/patern2/' file
```

Найти в файле строку с `text`, произвести замену по шаблону, сохранить старый файл как `file.bak`



Ответить



immaculate

28 апр 2017 в 20:38



По-моему, почти никто не использует `sed` и `awk` вне контекста `bash` (или другого `shell`). Так что я особого противоречия не вижу. `Bash` без утилит типа `grep` / `sed` / `awk` получается довольно ограниченным инструментом, ценность которого невелика.



Ответить



iig

28 апр 2017 в 20:49



Я вот ImageMagick тоже из под `bash` запускаю. И `gcc`. И `ssh`... :)



Ответить



ссссо
3 мая 2017 в 21:27

И при этом не понимаете разницы между bash и bash-скриптом. :)

↑ -2 ↓

Ответить

🔖 ...

iig
4 мая 2017 в 01:49

Даже так?

```
find -iname \*.bmp|xargs -l1 -i% convert "% " "%.png"
```

Это был bash или bash-скрипт? :)

↑ +1 ↓

Ответить

🔖 ...

ссссо
4 мая 2017 в 05:42

:)

Вот, Вы уже, как писали выше, запускаете ImageMagick из под bash. И gcc. И ssh. А, судя по вопросу, «плывёте» в элементарных вещах. Но не беда, я с удовольствием помогу Вам найти ответ на поставленный вопрос.

То, с чем Вы хотите разобраться – это команда для командной оболочки Unix Shell, такой как, например, bash (но не обязательно только для неё), которую (команду) она (оболочка) интерпретирует. Оболочка может делать это (интерпретировать команду) непосредственно из командной строки или (внимание!) из файла, представляющего собой скрипт командной оболочки.

Ответ на вопрос о том, сможет ли командная оболочка правильно интерпретировать указанную Вами команду, зависит от настройки Вашей системы и окружения. У Вас, например, может быть не установлен ImageMagick, или он установлен, но окружение, в контексте которого команда будет интерпретироваться, может ничего не знать о ImageMagick (по разным причинам).

Думаю, теперь Вы сами легко сможете ответить на поставленный Вами вопрос. Надеюсь, я смог Вам помочь.

:)

↑ -2 ↓

Ответить

🔖 ...

iig
4 мая 2017 в 10:57

Не каждый может коротко и ясно ответить на простой и понятный вопрос. Вернее, может, но не каждый. :)

Все же, та конструкция выше – это, по вашему, bash или bash-скрипт?

↑ +2 ↓

Ответить

🔖 ...

ссссо
4 мая 2017 в 15:10

:)

Не каждый может в дискуссии услышать ясный ответ собеседника на простой и понятный вопрос. Вернее, может, но не каждый. :)

(Подсказка: второй абзац текста моего ответа.)

;)



Ответить

**iig**

5 мая 2017 в 02:17

Есть такая DoS атака, dns amplification. Когда на короткий вопрос приходит длиинный ответ. ;)



Ответить

**cccco**

5 мая 2017 в 20:40

Есть такая логическая ошибка, называется «Мнимая логическая связь». Это когда ЖЕЛАЕМАЯ логическая связь выдаётся за истинную. Например, на КОРОТКИЙ вопрос должен быть только КОРОТКИЙ ответ. ;)



Ответить

**iig**

6 мая 2017 в 02:33

Хорошие ответы, как правило, короче вопросов. 42 например :)



Ответить

**cccco**

6 мая 2017 в 15:41

73



Ответить

**redfs**

4 мая 2017 в 12:26

Оболочка может делать это (интерпретировать команду) непосредственно из командной строки или (внимание!) из файла, представляющего собой скрипт командной оболочки.

Вообще говоря, вы сами или путаетесь или не до конца понимаете, о чём пишете.

Начать с того, что в unix – всё файлы. Даже «командная строка» (как вы говорите). Поэтому нет никакой принципиальной разницы, откуда шелл получает команды – из командной строки или из файла – для него это одно и то же.

С этой точки зрения пример выше – обычный скрипт использующий фильтр к потоку вывода.



Ответить

**cccco**

4 мая 2017 в 17:27

Вообще говоря, вы сами или путаетесь или не до конца понимаете, о чём пишете.

Судя по тому, что для доказательства своего утверждения (Ваше цитата выше), Вы привели следующую мою цитату:

Оболочка может делать это (интерпретировать команду) непосредственно из командной строки или (внимание!) из файла, представляющего собой скрипт командной оболочки.

правильно ли я понимаю, что Вы с этим моим утверждением не согласны и считаете его неправильным? Ну, тогда расскажите, пожалуйста, что неправильного в приведённом Вами моём утверждении, с которым Вы не согласны.

Я, в свою очередь, в приведённой Вами моей цитате не вижу утверждения о том, что для шелла есть принципиальная разница откуда получать команды. Наоборот, там как раз и говорится, что ему без разницы, откуда команды принимать: может от сюда, а может от туда — для него это одно и тоже.

Но, как и из моей приведённой Вами цитаты, так и из уже Вашей следующей цитаты:

Поэтому нет никакой принципиальной разницы, откуда шелл получает команды — из командной строки или из файла — для него это одно и то же.

мы можем сделать вывод, что шелл может получать команды из множества РАЗНЫХ мест. ("... нет никакой принципиальной разницы, откуда шелл получает команды...").

Так вот, bash-скрипт — это ВСЕГО ЛИШЬ одно из множество мест, ОТКУДА шелл получает команды. Причём место конкретное — КОМАНДНЫЙ файл.

Да, в unix всё — суть файлы. Но согласитесь, файлы-то могут быть разными. Например, есть бинарные, есть текстовые. Не из всех unix-файлов шелл может команды получить. Иначе мы можем прийти к выводу, что все в unix — суть скрипты шелла.

Не кажется ли Вам, что всё-таки это Вы:

... или путаетесь или не до конца понимаете, о чём пишете.



Ответить



 **cccco**
3 мая 2017 в 21:15

Как Вы думаете:

- bash и bash-скрипт — суть одно и тоже или нет?
- используется ли в bash-скриптах sed и awk?
- если в bash-скрипте используется sed и/или awk, остаётся ли он bash-скриптом или нет?



Ответить



 **redfs**
3 мая 2017 в 21:38

- bash и bash-скрипт — суть одно и тоже или нет?

Нет.

- используется ли в bash-скриптах sed и awk?

Может использоваться. Может не использоваться.

- если в bash-скрипте используется sed и/или awk, остаётся ли он bash-скриптом или нет?

Хм. В скрипте bash могут использоваться вызовы практически любых команд и программ, включая sed и awk. Если это скрипт bash, то он останется скриптом bash, перловкой он точно не станет.

Задавайте основной вопрос, можно без вступительной риторики :)



Ответить



 **cccco**
3 мая 2017 в 21:53

Основной вопрос — первый. :)



Ответить



 **iig**
28 апр 2017 в 19:29

При необходимости вывод `sed` можно перенаправить в файл, возможно – **перезаписать старый файл**

Хм. Я бы такого не советовал.

В данном случае в качестве разделителя использован восклицательный знак

В зависимости от того, в каких кавычках будет команда для `sed` (одинарные/двойные) – оно сработает по разному. Символы, которые `bash` интерпретирует по своему, лучше так не использовать.



Ответить



 **grossws**
28 апр 2017 в 19:52

Видимо, у `ruvds` дела идут столь хорошо, что промокод стал появляться в статье по два раза. И основная задача этого набора статей – публикация этого промокода и ссылки на свой сервис, а не публикация полезного материала, что, конечно, ожидаемо, но несколько печально.



Ответить



 **grossws**
28 апр 2017 в 19:55 

В нашем случае применена команда вида `s/pattern1/pattern2/`. Буква «s» – это сокращение слова «substitute», то есть – перед нами команда замены. `Sed`, выполняя эту команду, просмотрит переданный текст и заменит найденные в нём фрагменты (о том – какие именно, поговорим ниже), соответствующие `pattern1`, на `pattern2`.

Вводит читателя в заблуждение. Если посмотреть `sed(1)`, то там эта операция описана так: `s/regexp/replacement/`, что соответствует реальности. Таки `s///` заменяет не на другой `pattern/regexp`, а на `replacement string`, в котором могут использоваться подстановки на основе `capture group`.

Но какое отношение имеет `sed` к `bash` мне тоже не очень понятно. Так можно в статьях про баш и `perl/ruby/python` обсуждать.



Ответить



 **debsaw**
3 мая 2017 в 16:46

Для оформления правила обработки текста, заключённого в кавычки, используются прямые слэши

в качестве разделителя необязательно использовать слэши

```
$ echo 1214 | sed -e 's[1[5[g'  
5254
```



Ответить



 **Ivanq**
3 мая 2017 в 17:02

К счастью, sed позволяет нам самостоятельно задавать символы-разделители для использования их в команде замены. Разделителем считается первый символ, который будет встречен после s:

```
$ sed 's!/bin/bash!/bin/csh!' /etc/passwd
```

↑ 0 ↓ Ответить

 **Crystal_HMR**
3 мая 2017 в 18:03

При необходимости вывод sed можно перенаправить в файл, возможно – **перезаписать старый файл**.

```
wc -l .xsession-errors.old
#9165 .xsession-errors.old
sed 's/Indirect/REDIRECT/g' .xsession-errors.old > .xsession-errors.old
#
cat .xsession-errors.old
#
wc -l .xsession-errors.old
#0 .xsession-errors.old
```

А если бы я был новичком, а это был боевой конфиг?) Опять учите плохому.

↑ +1 ↓ Ответить

 **ghostinushanka**
3 мая 2017 в 19:00

Ваш пример к сожалению не объясняет сути граблей (а это на мой взгляд самое важное). Кому интересно – я уже писал почему это произойдёт вот тут.

↑ +1 ↓ Ответить

^ 1 v

Вы можете оставлять комментарии только к свежим публикациям

ЕЖЕДНЕВНЫЙ ХАБР | 21 СЕНТ 2026

 57K



Техдиректор Microsoft Azure с помощью ИИ за два дня портировал на macOS старую утилиту для Windows

Программист Microsoft Марк Руссинович, который в настоящее время является техническим директором Microsoft Azure, использовал искусственный интеллект, чтобы перенести инструмент Windows ZoomIt, который он поддерживает более 20 лет, на macOS за выходные.

В «Бюро 1440» протестировали спутниковый интернет в заполярном посёлке Нельмин - Нос в Ненецком автономном округе

Темная сторона эпохи перемен: как Геральт из Ривии сломал канон сказок

sip2go: Альтернатива SIP клиентам для Android

Ещё

0

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



ekaterinaredina

6 часов назад

Открываем AliceAI-Foundation-80B-A3B-Base – новую языковую модель Яндекса, обученную с нуля

81 мин

13K

+85

34

58



EmoRagnareks

15 часов назад

Скорость разработки кода перестала быть дефицитом. Дефицитом стали денежные идеи

Простой

5 мин

22K

Мнение

+29

36

127

**NuclearMaster**

4 часа назад

Безопасный источник энергии на тысячу лет. Ядерные реакторы IV поколения



Средний



6 мин



5.8K

Обзор



+27



8



16

**laska**

5 часов назад

Как за один Хакатон мы вживили ICQ в Битрикс24



Простой



8 мин



5K

Кейс



+20



8



4

**VMarkell**

22 часа назад

Почему умные NPC SkyrimNet ведут себя как попугаи? Эксперименты с ролеплейным ИИ



Простой



22 мин



12K

Мнение

Из песочницы



+20



10



21

**MegaCrash**

22 часа назад

sip2go: Альтернатива SIP клиентам для Android



Средний



6 мин



59K

Кейс

Из песочницы



+18



40



12

**Andrew_v_photo**

5 часов назад

А-ЦОД глазами инженера: зачем это дата-центру и что вообще там сертифицируют



9 мин



4.2K



+17



2



2

**beget_com**

6 часов назад

Лечим вывих по методу Латарже



Средний



20 мин



5.6K

Обзор



+17



7



7



Ekaterina_Siletskaya

3 часа назад

Как устроено автоматическое охлаждение данных в in-памяти HTAP СУБД:
разбор механизма на примере Tarantool Column Store

 9 мин

 4.2K

 +14

 1

 0



SLY_6

23 часа назад

На старом заводе в Питтсбурге бактерии научились питаться
промышленными отходами

 Простой

 6 мин

 9.8K

Перевод

 +12

 3

 2

Показать еще

ИНФОРМАЦИЯ

Сайт	ruvds.com
Дата регистрации	18 марта 2016
Дата основания	27 июля 2015
Численность	11–30 человек
Местоположение	Россия
Представитель	ruvds

ССЫЛКИ

- VPS / VDS сервер от 149 рублей в месяц
[ruvds.com](#)
- Дата-центры RUVDS в Москве, Санкт-Петербурге, Казани, Екатеринбурге, Новосибирске, Лондоне, Франкфурте, Цюрихе, Амстердаме, Измире
[ruvds.com](#)
- Помощь и вопросы
[ruvds.com](#)
- Партнерская программа RUVDS
[ruvds.com](#)
- VPS (CPU 1x2ГГц, RAM 512Мб, SSD 10 Gb) – 219 рублей в месяц
[ruvds.com](#)
- VPS Windows от 710 рублей в месяц. Попробуйте бесплатно на 3 дня
[ruvds.com](#)
- VDS в Цюрихе. Дата-центр TIER III – швейцарское качество по низкой цене.
[ruvds.com](#)

Антивирусная защита виртуального сервера. Легкий агент для VPS.

ruvds.com

VPS в Лондоне. Дата-центр TIER III – английская точность за рубли.

ruvds.com

VPS с видеокартой на мощных серверах 3,4ГГц

ruvds.com

ПРИЛОЖЕНИЯ

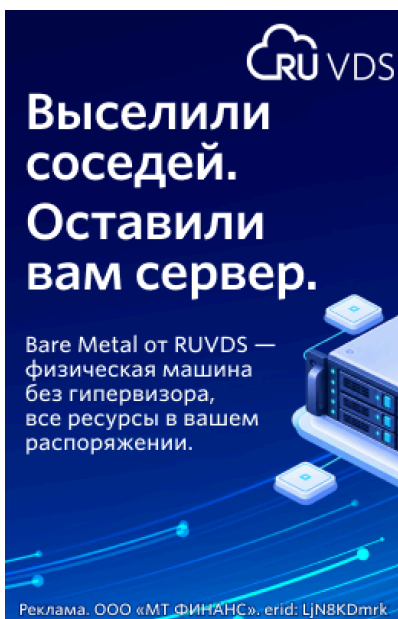


RUVDS Client

Приложение для мониторинга и управления виртуальными серверами RUVDS с мобильных устройств.

Android iOS

ВИДЖЕТ



БЛОГ НА ХАБРЕ

43 минуты назад

Чем занимаются Visa и Mastercard?

2K 0

4 часа назад

Безопасный источник энергии на тысячу лет. Ядерные реакторы IV поколения

5.8K 16

вчера в 20:01

Номерные радиостанции: «Венона» против «Энормоз», первая шпионско - криптографическая битва Холодной войны

13K 5

вчера в 16:01

Как мы сэкономили 100 ТБ за счёт оптимизации DNS-кэша сервиса 1.1.1.1

14K 4

19 сен в 20:01

Электрогитара «Ласточкин хвост», версия 2

 14K  4

Ваш аккаунт	Разделы	Информация	Услуги
Профиль	Статьи	Устройство сайта	Корпоративный блог
Трекер	Новости	Для авторов	Медийная реклама
Диалоги	Хабы	Для компаний	Нативные проекты
Настройки	Компании	Документы	Образовательные программы
ППА	Авторы	Соглашение	Стартапам
	Песочница	Конфиденциальность	



Настройка языка

Техническая поддержка

© 2006–2026, Хабр

На информационном ресурсе применяются [рекомендательные технологии](#)